Psytrance Generator

Architecture of an Energy-Driven Audio Synthesis Engine in Rust

Simon Knight

Adelaide, Australia

March 2026 • Version 1.0.0

Abstract. Psytrance is a terminal-based audio synthesis engine written in Rust that generates full psytrance tracks from declarative parameter configurations. The system combines manual oscillator synthesis, spectral sample analysis, and a hierarchical energy model to produce arrangement-aware electronic music in real time. This report presents the architecture of the synthesis pipeline, formalises the four-level energy model that drives pattern density and timbral evolution, describes the DSP chain (biquad filters, Schroeder reverb, ping-pong delay, sidechain compression), details the sample intelligence subsystem that classifies and blends audio samples with synthesised output, and documents the Ableton Live integration via OSC and Link. On an Apple M2 workstation the engine renders a 64-bar stereo track in 18 ms—approximately 470× real time—while maintaining 64-bit internal precision throughout the signal path.

Version	Date	Changes
1.0.0	March 2026	Initial release: 12 phases complete

Contents

List of Figures

List of Tables

1 Introduction

Psytrance is a subgenre of electronic dance music characterised by driving kick drums at 140–150 BPM, hypnotic bass lines built from filtered sawtooth oscillators, layered percussive textures, and carefully sculpted energy arcs that build tension and release over multi-minute arrangements. Producing a psytrance track in a conventional digital audio workstation (DAW) requires manually programming drum patterns, designing filter sweeps, tuning sidechain compression, and arranging dozens of clips across a timeline—a process that demands both musical knowledge and significant technical skill.

This report describes *Psytrance Generator*, a Rust-based synthesis engine that automates the core production workflow. The user specifies high-level parameters—tempo, mood, key, energy curve—and the engine generates a complete stereo mix with kick, bass, hi-hat, clap, lead, and sample layers, all shaped by a hierarchical energy model that controls pattern density, filter modulation, and sample placement across the arrangement.

The system is designed around three principles:

1. **Energy-driven composition.** A four-level energy model (macro curve, meso breathing, micro groove, sub-channel gating) determines *what* plays *when* and *how loudly*, replacing manual arrangement with a declarative energy specification.
2. **Hybrid synthesis.** Each musical element can be rendered from manual oscillators, from classified audio samples, or from a configurable blend of both—adapting the timbral palette to the energy state of the arrangement.
3. **DAW integration.** MIDI export, real-time OSC clip pushing via AbletonOSC, and Ableton Link transport synchronisation allow the generator to function as a compositional front-end for professional production workflows.

The remainder of this report is structured as follows. Section 2 presents the system architecture and module decomposition. Section 3 formalises the energy model. Section 4 describes the synthesis engines. Section 5 details the DSP processing chain. Section 6 covers the sample intelligence subsystem. Section 7 explains pattern generation. Section 9 documents the Ableton integration layer. Section 10 describes the terminal user interface. Section 11 evaluates rendering performance, and section 12 concludes with future directions.

2 Architecture

Psytrance Generator is implemented as a single Rust binary (edition 2021) comprising nine top-level modules. The crate compiles to approximately 12 MB on macOS and has no runtime dependencies beyond the system audio driver (CoreAudio on macOS, ALSA on Linux).

2.1 Module Decomposition

Figure 1 illustrates the module dependency graph. Data flows left to right: the `theory` module provides musical primitives (keys, scales, moods) to the `engine`, which orchestrates pattern generation and calls into `synths` and `samples` for audio rendering. The `dsp` module provides shared signal processing primitives consumed by all synthesis code. The `output` module handles WAV export, MIDI export, and real-time playback. The `osc` and `link` modules manage Ableton integration, and the `tui` module provides the interactive terminal interface.

2.2 Key Data Structures

The system is organised around three central data structures:

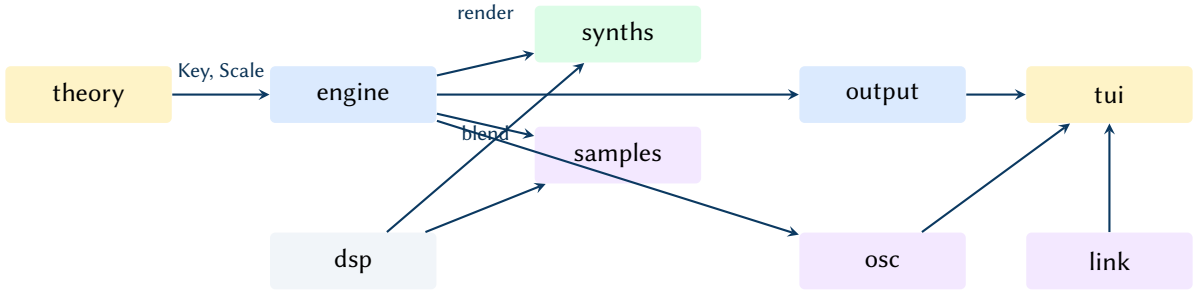

Figure 1. Module dependency graph. Colours indicate functional layers: gold for music theory, blue for core engine and output, green for synthesis, purple for integration and samples, grey for DSP primitives.

Figure 2. The seven-stage render pipeline.

- `GeneratorConfig` — the single source of truth for all generation parameters (BPM, key, mood, filter settings, energy curve, mute/solo state, pattern presets, humanisation level). Serialisable via `serde` for session persistence.
- `EnergyModel` — the hierarchical energy calculator that maps a bar number to per-element energy levels (section 3).
- `RenderResult` — the immutable output of a render pass, containing interleaved stereo samples, per-element stems, bar pattern metadata for visualisation, and MIDI export data.

> **Design Decision 1: Single Source of Truth**
>
> `GeneratorConfig` flows unmodified from CLI parsing through the TUI state into the `Generator`. Any parameter change in the TUI triggers a full re-render from the config, ensuring that the audio output always exactly reflects the visible parameter state. This design eliminates an entire class of state-synchronisation bugs at the cost of a re-render on every change—a cost that is negligible at 18 ms per 64-bar track (section 11).

2.3 Render Pipeline

Figure 2 shows the complete render pipeline. The `Generator::render()` method executes the following stages:

1. **Energy evaluation** — query the `EnergyModel` for each bar to obtain per-element energy levels.
2. **Pattern generation** — produce `NoteEvent` sequences for kick, bass, hi-hat, clap, and lead using energy-driven density and preset-specific algorithms.
3. **Audio synthesis** — render each note event to a per-element mono buffer using either oscillator synthesis, sample playback, or a weighted blend of both.
4. **Sidechain compression** — duck the bass signal using a kick-triggered envelope follower.
5. **Stereo mixdown** — pan elements using constant-power panning and sum to a stereo bus.
6. **Effects** — apply ping-pong delay (hi-hat/clap/lead bus), Schroeder reverb (master bus), and a 30 Hz master HPF.
7. **Limiting** — normalise to −0.5 dBFS peak and apply a tanh-based soft-clip limiter.

3 The Energy Model

The energy model is the central organising abstraction of the generator. Rather than manually arranging patterns on a timeline, the user defines an energy curve and the model determines pattern density, filter modulation depth, sample selection, and element entry/exit timing. The model operates at four hierarchical levels.

3.1 Macro Curve

The macro curve defines the overall energy arc of the track as a piecewise-linear function $E_{\text{macro}}(b)$ over bar number b. It is specified as a sequence of control points $(b_i, e_i) \in [0, B] \times [0, 1]$ where B is the total number of bars:

$$E_{\text{macro}}(b) = e_i + (e_{i+1} - e_i) \cdot \frac{b - b_i}{b_{i+1} - b_i}, \quad b_i \le b < b_{i+1} \tag{1}$$

Four named presets provide common arrangement shapes:

- **build_and_drop** — rises to 0.9 at 30%, drops to 0.3 at 35%, climbs to 1.0 at 70%, falls to 0.2 at end.
- **full_set** — monotonic rise from 0.2 to 1.0 at 75%, then a gentle decline to 0.7.
- **wave** — oscillating peaks at 0.6, 0.8, 0.9, 1.0 with valleys between, creating a rolling energy feel.
- **tension_release** — two sharp drops (0.7 → 0.2 at 30%, 0.9 → 0.3 at 60%) with recovery climbs, resolving to 1.0 at 90%.

3.2 Meso Breathing

The meso pattern imposes a phrase-level breathing rhythm as an 8-element multiplier array $M = [m_0, \ldots, m_7]$ that repeats every 8 bars. The default *breathing* pattern is:

$$M_{\text{breathing}} = [0.85,\ 0.90,\ 0.95,\ 1.0,\ 1.0,\ 0.95,\ 0.90,\ 0.85] \tag{2}$$

The combined energy at bar b is $E(b) = \text{clamp}(E_{\text{macro}}(b) \cdot M[b \bmod 8],\ 0,\ 1)$.

3.3 Micro Groove

The micro groove shapes velocity at the 16th-note level within each bar. A 16-element array $G = [g_0, \ldots, g_{15}]$ multiplies the velocity of each step, creating rhythmic accents. The *psy_groove* preset emphasises downbeats and offbeat drive:

$$G_{\text{psy}} = [1.0,\ 0.6,\ 0.75,\ 0.5,\ 0.9,\ 0.55,\ 0.8,\ 0.5,\ 0.95,\ 0.6,\ 0.75,\ 0.5,\ 0.85,\ 0.55,\ 0.8,\ 0.65] \tag{3}$$

3.4 Sub-Channel Gating

Each of the five musical elements (kick, bass, hi-hat, clap, lead) has an independent sub-channel that gates and shapes the element's response to the macro energy. A sub-channel is defined by four parameters:

The effective energy level for element i at bar b is:

$$E_i(b) = \begin{cases} 0 & \text{if } b < \text{onset}_i \text{ or } E(b) < \text{thresh}_i \\ \left(\min\left(1,\ \frac{b - b_{\text{active}}}{\text{attack}_i}\right) \cdot E(b)\right)^{\gamma_i} & \text{otherwise} \end{cases} \tag{4}$$

Table 1. Default sub-channel parameters. The response curve exponent controls how aggressively the element responds to energy changes: < 1 gives a gentle ramp, > 1 gives a sharp threshold effect.

Element	Onset Bar	Threshold	Attack (bars)	Response γ
Kick	0	0.05	2	0.7
Bass	4	0.15	4	1.0
Hi-hat	8	0.25	4	1.2
Clap	8	0.35	2	1.5

where b_{active} is the first bar at which both conditions (onset reached and threshold exceeded) are satisfied. The attack ramp fades the element in over attack_i bars after activation, and the response exponent γ_i shapes the energy-to-density transfer function.

Design Decision 2: Staggered Entry

The sub-channel onset and threshold parameters create a natural staggered entry: kick appears first (bar 0), bass enters at bar 4 once energy exceeds 0.15, and percussion layers arrive at bar 8. This mirrors the arrangement conventions of psytrance production, where elements are introduced progressively during the intro and build sections.

4 Synthesis Engines

The generator includes four dedicated synthesis engines, each implementing the specific timbral characteristics of its target element. All synthesis operates in `f64` precision; conversion to `f32` occurs only at the final output stage.

4.1 Kick Drum

The kick drum synthesiser produces up to 300 ms of audio per hit by summing three layers:

1. **Main body** — a sine oscillator with an exponential pitch envelope sweeping from $f_{\text{start}} \approx 150$ Hz to $f_{\text{end}} \approx 50$ Hz. The instantaneous frequency is:

$$f(t) = f_{\text{end}} + (f_{\text{start}} - f_{\text{end}}) \cdot e^{-\alpha_p t} \tag{5}$$

   where $\alpha_p \approx 35$ controls the pitch decay rate. The amplitude envelope is $a(t) = e^{-\alpha_a t}$ with $\alpha_a \approx 8$.
2. **Sub-harmonic** — a 25 Hz sine with a slower decay ($\alpha_{\text{sub}} \approx 2.5$) at 35% of the main body level. This extends the low-frequency sustain without muddying the transient.
3. **Click transient** — a short (~3 ms) burst of a high-frequency sine at ~3500 Hz with a linear fade-out. The click provides the initial attack snap that helps the kick cut through the mix.

Phase Accumulation

All oscillators in the generator use phase accumulation ($\phi \mathrel{+}= f/f_s$) rather than direct computation ($\sin(2\pi f t)$). This is essential for the kick's pitch sweep: recomputing $\sin(2\pi f(t) \cdot t)$ with a time-varying $f(t)$ produces phase discontinuities that manifest as audible clicks at the sweep inflection points.

4.1.1 Humanisation

The `humanize` parameter (0–1) scales per-hit random variation across all three layers. The humanised parameters and their variation ranges at full humanisation are:

- Start frequency: ±12 Hz
- End frequency: ±3 Hz
- Pitch decay rate: ±4
- Click frequency: ±400 Hz
- Click level: ±8%
- Sub level: ±7%

4.2 Bass Synthesiser

The bass engine renders one note at a time with a subtractive synthesis architecture: two detuned oscillators into a modulated biquad filter, followed by waveshaping and an ADSR amplitude envelope.

4.2.1 Oscillator Section

Two oscillators run simultaneously: a main oscillator at the note frequency f_0 and a detuned oscillator at $f_0 \cdot 2^{7/1200}$ (+7 cents). Each oscillator can be either a naive sawtooth (phase accumulation) or a wavetable oscillator scanning through a loaded wavetable. The mix ratio is 70% main, 30% detuned, creating a chorus-like thickening effect.

4.2.2 Filter Modulation

The biquad filter cutoff is modulated per-sample by three sources:

$$f_c(n) = f_{\text{base}} + \underbrace{E_{\text{filt}}(n) \cdot A_{\text{env}}}_{\text{envelope}} + \underbrace{L(n) \cdot D_{\text{lfo}}}_{\text{LFO}} \tag{6}$$

where $E_{\text{filt}}(n)$ is the filter envelope output (AD shape with configurable attack, decay, sustain, and amount in Hz), and $L(n)$ is the LFO output (unipolar, 0–1) with depth D_{lfo} in Hz. The filter's `set_params(`f_c`,` Q`)` method recomputes the RBJ biquad coefficients at every sample—a deliberate design choice that trades computational cost for artifact-free modulation.

> **Design Decision 3: Per-Sample Coefficient Updates**
>
> Updating biquad coefficients per-sample is unusual in production audio engines, which typically update at block boundaries (every 32–128 samples) and interpolate. We chose per-sample updates because the psytrance genre features extremely fast filter sweeps (acid-style envelopes with 0 ms attack and sub-200 ms decay) where block-rate updates would produce audible zipper noise. At our target sample rate of 44.1 kHz, the coefficient computation (two trigonometric functions and a handful of multiplications) adds approximately 4% to the bass rendering time—an acceptable cost given the 470× real-time headroom (section 11).

Four filter envelope presets are provided:

Table 2. Filter envelope presets. The *amount* parameter specifies the peak cutoff offset in Hz above the base frequency.

Preset	Attack (ms)	Decay (ms)	Sustain	Amount (Hz)
Off	—	—	—	0
Acid	0	180	5%	2000
Slow Sweep	50	500	20%	1500
Pluck	0	80	0%	3000

4.2.3 Waveshaping and Envelope

The filtered signal is passed through a tanh waveshaper: $y = \tanh(1.3x)$, which adds odd harmonics and provides soft saturation. The amplitude envelope is a simple ADSR with fixed segments: 3 ms linear attack, 40 ms exponential decay to 70% sustain, sustain hold for the note duration, and 30 ms linear release.

4.3 Percussion

4.3.1 Hi-Hat

Hi-hats combine two sources: white noise filtered through a biquad HPF, and a metallic sine tone at approximately 7500 Hz. Closed hi-hats use a 7000 Hz HPF cutoff and fast amplitude decay (e^{-100t}, ~30 ms effective duration). Open hi-hats use a lower 6000 Hz cutoff and slower decay (e^{-15t}, ~150 ms).

4.3.2 Clap

The clap synthesiser produces a multi-tap noise envelope: three consecutive 10 ms bursts at 15 ms spacing (attenuating at 100%, 80%, 60%), followed by a 100 ms filtered noise tail with exponential decay. A 1000 Hz HPF removes low-frequency content. The multi-tap structure simulates the "multiple hands clapping" effect that gives the sound its characteristic scattered attack.

4.4 Lead (FM Synthesis)

The lead synthesiser implements two-operator FM synthesis:

$$y(t) = A(t) \cdot \sin\left(2\pi f_c\, t + \underbrace{I \cdot E_{\text{fm}}(t) \cdot \sin(2\pi f_m\, t)}_{\text{modulator}}\right) \tag{7}$$

where f_c is the carrier frequency (note pitch), $f_m = f_c \cdot R$ is the modulator frequency (R is the FM ratio), I is the modulation index, and $E_{\text{fm}}(t) = e^{-5t}$ is a fast FM envelope that creates a time-varying timbre. The modulation index I and carrier amplitude decay are scaled by the macro energy level: higher energy produces brighter, longer lead tones.

5 DSP Processing Chain

The `dsp` module provides shared signal processing primitives used by all synthesis engines and the master effects chain. Figure 3 illustrates the signal flow through the effects stage.

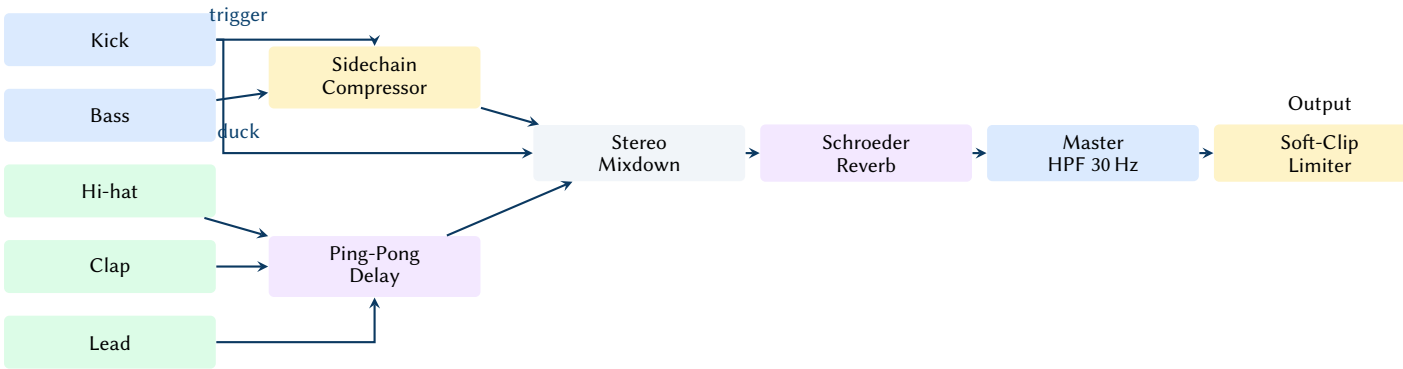

Figure 3. Master effects signal flow. The kick triggers the sidechain compressor which ducks the bass. Hi-hat, clap, and lead pass through the ping-pong delay before joining the stereo bus.

5.1 Biquad Filter

The `Biquad` struct implements a second-order IIR filter using the RBJ Audio EQ Cookbook [1] formulas. It supports four filter types: low-pass, high-pass, band-pass, and notch. The filter processes samples using Direct Form II Transposed:

$$y[n] = b_0 x[n] + b_1 x[n-1] + b_2 x[n-2] - a_1 y[n-1] - a_2 y[n-2] \tag{8}$$

Stability is maintained by clamping the cutoff frequency to $[20,\ 0.45 f_s]$ Hz and the Q factor to $[0.5, 20]$. The upper cutoff clamp at $0.45 f_s$ (rather than $0.5 f_s$) prevents numerical instability as ω_0 approaches π.

5.2 Sidechain Compression

The sidechain compressor ducks the bass signal using the kick's amplitude envelope as a control signal:

$$g_{\text{sc}}[n] = 1 - d \cdot \min(3 \cdot s[n],\ 1) \tag{9}$$

where $d = 0.75$ is the duck depth and $s[n]$ is the kick envelope follower with instantaneous attack and exponential release:

$$s[n] = \begin{cases} |x_{\text{kick}}[n]| & \text{if } |x_{\text{kick}}[n]| > s[n-1] \\ s[n-1] \cdot \alpha_r & \text{otherwise} \end{cases} \tag{10}$$

where $\alpha_r = e^{-1/(0.12 f_s)}$ gives a 120 ms release time constant.

5.3 Ping-Pong Delay

The `PingPongDelay` uses two circular buffers (left and right) with cross-feedback routing and a one-pole IIR low-pass filter in each feedback path for analog-style warmth:

$$\text{buf}_L[w] = x_L + \text{fb}_R \cdot k \tag{11}$$

$$\text{buf}_R[w] = x_R + \text{fb}_L \cdot k \tag{12}$$

where k is the feedback coefficient (clamped to 0.95 maximum) and w is the write position. The delay time is tempo-synchronised: a dotted 16th note at the current BPM, computed as $t_d = 0.375 \cdot 60/\text{BPM}$ seconds.

Default parameters: 35% feedback, 60% damping coefficient, 20% wet mix.

5.4 Schroeder Reverb

The `StereoReverb` implements the classic Schroeder topology [2] with decorrelated channels:

1. **Four parallel comb filters per channel** (eight total). Delay lengths at 44.1 kHz are chosen to be mutually prime to prevent flutter echo: left channel uses $\{1116, 1188, 1277, 1356\}$ samples; right channel uses $\{1139, 1211, 1300, 1379\}$. A one-pole low-pass filter in each feedback path provides frequency-dependent damping.
2. **Two series allpass filters per channel.** Delays: left $\{556, 441\}$, right $\{579, 464\}$. The allpass coefficient $g = 0.5$.

3. **Wet/dry mix** at 12% wet.

Default parameters: decay coefficient 0.4, damping 0.7, giving an approximate RT60 of 0.6 s.

5.5 Master Processing

Two final processing stages are applied to the stereo mix:

- **30 Hz high-pass filter** — a second-order Butterworth (biquad HPF, $Q = 0.707$) removes subsonic energy accumulated from the kick's sub-harmonic layer and bass oscillator DC drift.
- **Soft-clip limiter** — normalises to −0.5 dBFS peak, then applies a tanh soft clipper with a knee at 0.9:

$$y = \begin{cases} x & |x| \leq 0.9 \\ \text{sign}(x) \cdot \left(0.9 + 0.1 \cdot \tanh\left(\frac{|x|-0.9}{0.1}\right)\right) & |x| > 0.9 \end{cases} \tag{13}$$

6 Sample Intelligence

The sample intelligence subsystem analyses, classifies, and blends audio samples with the synthesised output. It operates in four stages: spectral analysis, rule-based classification, cached profile storage, and energy-aware placement.

6.1 Feature Extraction

The `analyzer` module extracts seven acoustic features from each loaded WAV file using `rustfft` [3] for spectral analysis:

Table 3. Extracted acoustic features per sample.

Feature	Unit	Method
Spectral centroid	Hz	Energy-weighted mean frequency from Hann-windowed FFT ($\bar{f} = \sum f_i \lvert X_i \rvert / \sum \lvert X_i \rvert$)
RMS level	0–1	Root mean square amplitude
Zero crossing rate	0–1	Sign change count normalised by sample length
Attack time	ms	Time to reach 90% of peak RMS (5 ms analysis frames)
Tonal score	0–1	Normalised autocorrelation peak over lags 1 to $N/2$
Amplitude slope	—	Linear regression slope of per-frame RMS (positive = riser)
Brightness	0–1	Normalised centroid: $\min(1, \text{centroid}/10000)$

6.2 Classification

The `classifier` module applies a rule-based decision tree to assign each sample to one of nine categories: `Kick`, `Snare`, `Hat`, `Clap`, `Bass`, `Fx`, `Riser`, `Texture`, `Vocal`.

```
fn classify(profile: &SampleProfile, hint: Option<SampleCategory>)
    -> SampleCategory
{
    let p = profile;
```

```
    if p.spectral_centroid < 300.0
        && p.zcr < 0.05
        && p.attack_ms < 50.0
    {
        return SampleCategory::Kick;
    }
    if p.duration_ms > 2000.0 && p.amplitude_slope > 0.0 {
        return SampleCategory::Riser;
    }
    if p.spectral_centroid > 3000.0 && p.zcr > 0.15 {
        return SampleCategory::Hat;
    }
    // ... additional rules omitted for brevity
    hint.unwrap_or(SampleCategory::Fx)
}
```

When the rule-based classifier cannot determine a category, it falls back to a directory-name hint (e.g. a sample in a `kicks/` subdirectory is classified as `Kick`).

6.3 Profile Caching

Computed profiles are stored as sidecar JSON files (`<stem>.sample.json`) alongside the source WAV. Cache validity is checked against the source file's modification timestamp and a schema version counter, ensuring that profiles are automatically recomputed when the analyser is updated.

6.4 Energy-Aware Placement

The `PlacementScheduler` pre-computes per-bar placement decisions for non-drum sample categories (FX hits, risers, textures, fills, vocals) based on the macro energy curve's value and slope:

- **Risers** — placed at 4-bar boundaries when the energy slope exceeds 0.15, with length chosen from $\{4, 8, 16\}$ bars to match the estimated climb duration.
- **FX hits** — at 4-bar boundaries when energy > 0.80; additionally at 2-bar boundaries when energy > 0.90.
- **Textures** — on energy plateaus ($|\text{slope}| < 0.05$) above 0.30 energy, at 4/8/16-bar intervals depending on density.
- **Fills** — at 4-bar boundaries above threshold, always at 8 and 16-bar boundaries.
- **Vocals** — every 16 bars, outside the 0.4–0.8 energy range (appearing in intros and peaks but not mid-energy builds).

6.5 Synth/Sample Blending

Each element's output can be pure synthesis, pure sample playback, or a weighted blend. The default blend ratio is computed from the macro energy level:

$$r_{\text{blend}} = \text{clamp}(0.8 \cdot E_{\text{macro}},\ 0,\ 0.8) \tag{14}$$

At low energy, synthesis dominates (clean, controlled sound); at high energy, samples are blended in (adding the organic variation and complexity of real recordings). Per-element blend overrides are available in the TUI.

7 Pattern Generation

All pattern generators take an `EnergyState` (the combined output of the energy model for the current bar) and produce a `BarPattern` containing a vector of `NoteEvent` structs:

```rust
pub struct NoteEvent {
    pub step: u8,              // 0..15 (16th note grid)
    pub velocity: f64,         // 0.0..1.0
    pub pitch_hz: f64,         // fundamental frequency
    pub duration_steps: u8,    // note length in 16ths
    pub timing_offset_samples: i32,  // micro-timing humanisation
}
```

7.1 Energy-Driven Density

Pattern density scales with the element's energy level. For bass, the number of active steps per bar is:

$$N_{\text{steps}} = \text{round}(4 + E_{\text{bass}} \cdot 12) \tag{15}$$

yielding 4 steps at zero energy (quarter notes) and 16 steps at full energy (continuous 16th notes).

7.2 Presets and Variation

Each element has multiple pattern presets that define distinct rhythmic characters. Table 4 summarises the kick presets as a representative example.

Table 4. Kick pattern presets. Each preset defines placement rules that are further modulated by the energy level.

Preset	Description
Default	Beats 1 and 3 always; beats 2 and 4 if energy > 0.3; ghost 16ths if > 0.7; stochastic pickups and flams
Minimal	Beats 1 and 3 only
Driving	All four beats plus probabilistic 16th-note fills
Breakbeat	Syncopated placement at steps $\{0, 3, 6, 10, 14\}$

7.3 Micro-Timing Humanisation

Three layers of humanisation are applied, all scaled by the global `humanize` parameter (0–1):

1. **Velocity jitter** — ±12%, with softer hits varying more: jitter $= 1 \pm 0.12 \cdot (1 - v \cdot 0.5)$.
2. **Micro-timing** — downbeats are displaced by ±1 ms; offbeats by ±3.5 ms. At 44.1 kHz, 3.5 ms is approximately 154 samples.
3. **Timbral variation** (kick only) — per-hit variation in pitch envelope, click frequency, and sub-harmonic level (section 4.1).

> **MIDI Preservation**
>
> Micro-timing offsets are preserved in MIDI export. At 960 PPQ, one tick is approximately 0.44 ms at 142 BPM, so a ±3.5 ms offset maps to ±8 MIDI ticks—sufficient resolution to reproduce the humanisation feel in a DAW.

8 Eastern Musical Traditions in Electronic Music

While the initial design targeted classic Western modes and 16-step psytrance syncopation, the architecture generalises cleanly to other musical traditions. This project includes raga-inspired scales and tala-inspired accent cycles that can be applied without changing the global time signature.

8.1 Raga-Inspired Scale Sets

The `theory` layer exposes additional scale interval sets representing common raga colours (e.g., Bhairav, Todi, Malkauns). In 12-TET these are approximations rather than strict microtonal renderings; nevertheless they provide a strong melodic fingerprint. In practice, these scales are most effective when paired with slower filter motion and less harmonic density, allowing the characteristic intervals to remain perceptible.

8.2 Tala Accent Cycles as Syncopation

Tala cycles such as Rupak (7), Dadra (6), and Keherwa (8) are modelled as 16-step velocity multiplier patterns. The engine retains a 4/4 bar grid, but applies tala-derived accents to create cross-rhythmic tension. This approach yields the sensation of a longer underlying cycle without requiring polymetric transport.

8.3 Yoga Flow as Energy Architecture

In addition to DJ-style build/drop curves, macro energy curves include a `yoga_flow` arc: a low-energy opening, warm-up, sustained work phase, a single peak, and a long cool-down into near-silence. This structure pairs naturally with raga colour and supports meditative outputs.

8.4 Motif Foreshadowing via Sub-Channel Preview

Sub-channel gating includes a preview window below the main activation threshold. In this region elements can appear as sparse "ghost" hits, hinting at future motifs before the full density arrives. The result is a stronger sense of narrative continuity: motifs are introduced, developed, and then brought into the foreground.

8.5 Drone, Ornamentation, and Cadential Devices

Beyond interval sets and accent cycles, Indian classical identity is often carried by a small set of structural devices:

- **Tanpura drone** — a continuous pitch reference (Sa–Pa–Sa) that anchors intonation and fills sparse sections. The engine can render a four-string drone layer tuned to the active key and mixed inversely with macro energy.
- **Aroh/avroh constraints** — ragas are not only scales; they encode preferred ascending and descending motion. A raga-aware melody generator uses directional step sets and avoids stepwise motion that contradicts the raga's character.
- **Vadi/samvadi weighting** — note hierarchy influences phrase landing points and perceived emotional center. The melodic generator can weight pitch selection toward the vadi (primary) and samvadi (secondary) degrees.
- **Gamaka** — ornaments such as meend (glide), kan-swar (grace approach), and andolan/kampita (oscillation) provide micro- expression that distinguishes ragas even in 12-TET approximations.
- **Tabla bols and articulation** — percussive syllables (e.g., Dha, Tin, Ghe) encode stroke families and resonance. The tom layer can attach articulation metadata to events and render different envelopes/spectra per stroke.
- **Tihai and korvai** — cadential patterns that resolve on sam (downbeat). These devices can be inserted at section boundaries to increase formal coherence and create a clear sense of arrival.

8.6 Time, Notation, and Breath

Two additional ideas extend the system beyond purely pattern-driven EDM generation:

- **Samay** — time-of-day associations can bias raga selection toward traditional prahar groupings.

- **Sargam + bols in the TUI** — displaying Sa/Re/Ga/Ma or bol initials in the step grid provides a musically meaningful view of eastern-mode patterns.
- **Breath-synced modulation** — a slow asymmetric LFO can modulate global dynamics and timbre (filter cutoff, wetness), and reduce high-frequency density during exhale to support meditative outputs.

8.7 Planned Extensions

The same architecture supports additional eastern concepts as future work: shruti-based microtonal offsets (with a purity parameter), an Alap–Jor–Gat performance arc that sequences feature activation, and a high-level rasa (aesthetic essence) framework that maps emotion to synthesis and arrangement parameters.

8.8 Implemented Music-Theory Features (Eastern Layer)

The eastern layer is not solely conceptual: it is implemented across the generator configuration, theory primitives, and TUI interaction model. This section documents the concrete features that currently ship, how they are represented in code, and how they influence rendering.

Raga Scale Sets and Phrase Seeds Raga-coloured scale sets are defined in the `theory` layer as interval lists (12-TET semitone offsets) with optional phrase metadata in `src/theory/scales.rs`. In addition to a scale *name* and *intervals*, selected ragas expose a *pakad* structure: a characteristic phrase plus explicit ascending/descending degree sequences. This enables two distinct uses:

- **Constraint** — all pitch selection is constrained to the active scale.
- **Identity injection** — pakad phrases can be injected as melodic seeds, giving a recognisable contour beyond raw interval choice.

Pakad Engine (Phrase-Based Melody Generation) When a selected scale provides a pakad, the engine can instantiate a `PakadEngine` (`src/engine/pakad.rs`) that cycles through a small phrase bank (characteristic, ascending, descending). For each bar it generates a sparse-to-dense pattern as a function of macro energy:

- Density increases with energy (fewer notes at low energy; more notes at high energy).
- Step stride shortens as energy rises (quarter-note feel $\rightarrow$ 8ths $\rightarrow$ 16ths).
- A controlled variation mode perturbs degrees by ± 1 with probability increasing above mid energy, while remaining in-scale.

This mechanism is intentionally lightweight: it preserves the generator's parameterised design while allowing raga identity to emerge from recurring phrase contours.

Alankar Generator (Permutation Exercises as Ornamented Runs) The generator includes an `AlankarGenerator` (`src/engine/alankar.rs`) that emits bar-length note patterns derived from alankar-style degree permutations. Implemented variants include Sequential, Skip, Mirror, Spiral, Zigzag, and Vakra (a constrained random walk). The generator exposes:

- **Type** — selects the permutation strategy.
- **Window size** — controls the size of the scale-degree window used by some strategies.
- **Direction** — ascending, descending, or stochastic.
- **Speed (laya)** — maps to steps-per-note (e.g., slower at low energy; faster at high energy).

In the current composition logic the alankar layer is treated as a high-energy embellishment: it can take over lead generation above an energy threshold to produce fast, raga-constrained runs without requiring explicit note programming.

Bol Text Input (Tabla Syllables to Step Grid) The TUI provides a bol text entry mode for quickly imposing a tala/tabla-inspired rhythm onto the tom/percussion lane (`src/tui/app.rs`). A bol string is parsed by the bol parser (`src/engine/bol_parser.rs`) into tokens; each token can carry an articulation label and a velocity. Tokens are then mapped onto a 16-step bar by wrapping or truncation. The resulting per-step overrides are written into the next bar's pattern override state (steps, velocity, and articulation arrays) via the step-edit override structures in `src/engine/generator.rs`.

This feature provides a human-readable rhythmic interface: rather than editing 16 booleans, a user can type a mnemonic sequence and immediately audition it in context.

Shruti Purity (Microtonal Blend) Microtonal behaviour is exposed as a continuous `shruti_purity` parameter in the generator configuration (`src/engine/generator.rs`).

- **0.0** — strict 12-TET tuning.
- **1.0** — apply the full raga-specific shruti offset table (when available for the active scale).

At render time, note frequencies are derived via a tuning function *after* degree selection: the melodic logic remains scale-degree based, while the final oscillator frequencies can slide microtonally. This separation keeps pattern generation deterministic and allows shruti to be treated as a timbral/intonational layer.

Raga Modulation (Scale Transition Over Bars) The engine supports raga modulation as a scheduled transition from a source raga to a target raga over `transition_bars` beginning at `start_bar` (`src/engine/raga_modulation.rs`). The modulation state exposes a progress function $t \in [0, 1]$ for any bar index.

In practice, modulation is implemented as scale blending: common tones are favoured early in the transition, while notes unique to the target scale are introduced as t increases. This enables mid-track colour changes without abrupt key changes or DAW-side automation.

Time-of-Day (Samay) Mood Selection The `samay` mood preset (`src/theory/mood.rs`) selects a raga and an energy curve based on local time-of-day (hour buckets). This couples two orthogonal parameters:

- **Scale choice** — biases toward ragas traditionally associated with certain times.
- **Energy architecture** — selects among curves such as `yoga_flow`, `long_journey`, and `ambient_meditation` to match the intended affect.

This feature demonstrates the intended design philosophy: music theory is encoded as deterministic selection logic over user-facing parameters, not as opaque learned behaviour.

Breath/Chakra/Arc Controls (Theory-Inspired Macro Behaviour) Several additional switches connect theory-inspired concepts to synthesis and arrangement:

- **Breath-synced LFO** — a slow modulation source that can shape brightness and wetness, particularly effective for low-energy outputs.

- **Chakra mode** — a global mode that maps sections/energy to chakra-derived colour and parameter bias.
- **Performance arc** — an Alap/Jor/Gat-inspired gating mode that controls feature activation over time.

These controls do not replace the energy model; rather, they provide interpretable, named presets that guide how energy maps onto timbre and pattern density.

User-Facing Controls in the TUI The eastern layer is directly manipulable in the terminal UI (`src/tui/app.rs` and `src/tui/ui.rs`). Notable bindings include:

- **Bol entry** — `:` enters bol input mode; on submit the bol string is applied to the next bar's tom lane.
- **Raga modulation** — a shifted key binding cycles through eastern ragas and schedules a transition over a fixed bar window.
- **Alankar** — a shifted key binding cycles the alankar type (and enables it); an alternate-modifier binding toggles alankar on/off.
- **Shruti purity** — incremental adjustments blend between 12-TET and shruti offsets.
- **Samay** — a dedicated binding applies the time-of-day recommendation (mood preset swap).

Because the generator treats the configuration as the single source of truth (section 2), each toggle results in a deterministic re-render that updates both the audio output and the visible state.

Feature-to-Implementation Map Table 5 summarises the primary entry points for each implemented feature: the configuration field that carries it, the module that implements it, and the principal render-time effect.

Configuration Surface (Eastern Parameters) The eastern layer is exposed as explicit fields on `GeneratorConfig` (`src/engine/generator.rs`). This makes the feature set accessible from the CLI, from preset serialization (`serde`), and from the TUI re-render loop.

Table 6. Eastern-layer configuration fields in `GeneratorConfig`.

Field	Type	Meaning / Notes
`mood_name`	`String`	Selects a mood preset (e.g., `"eastern"`, `"meditative"`, `"devotional"`, `"samay"`); mood determines key/scale, energy curve name, and filter ranges.
`key_override`	`Option<(u8,String)>`	Optional override of the mood's root MIDI note and scale name (e.g., `(38, "bhairav")`); applied when resolving the mood.
`energy_curve_override`	`Option<String>`	Overrides the macro curve preset name (e.g., `"yoga_flow"`, `"long_journey"`, `"ambient_meditation"`); used by `MacroCurve::from_name`.

Continued on next page

Field	Type	Meaning / Notes
`drone_enabled`	`bool`	Enables tanpura-style drone synthesis layer; typically mixed inversely with macro energy to support sparse/meditative sections.
`breath_lfo_enabled`	`bool`	Enables breath-synced modulation (slow LFO) to shape brightness/wetness; useful for yoga/ambient curves.
`chakra_mode`	`bool`	Enables chakra-derived parameter bias and/or visual mapping in the TUI; can also be implied by yoga-style energy presets.
`shruti_purity`	`f64`	Microtonal blend in $[0, 1]$: `0.0` = 12-TET; `1.0` = full shruti offsets (when the active scale provides a tuning map).
`performance_arc_mode`	`Option<ArcMode>`	Enables Alap/Jor/Gat-inspired gating of features over time; influences when melodic density and layers become available.
`modulation`	`Option<RagaModulation>`	Scheduled source→target raga transition: `start_bar`, `transition_bars`, and progress function t used at render time for scale blending.
`alankar_enabled`	`bool`	Enables alankar-derived melodic generation as a lead-mode enhancement (typically above an energy threshold).
`alankar`	`AlankarGenerator`	Alankar configuration (type, window size, direction, speed/laya, cursor state); serialized so repeated renders continue the pattern evolution.
`tani_trigger_bar`	`Option<u32>`	Schedules a percussion-solo style event (tani avartanam) beginning at a bar index; used as an arrangement accent.
`jugalbandi_mode`	`JugalbandiMode`	Selects call-and-response behaviour between voices; interacts with melody/pattern generation rather than tuning directly.
`layakari_enabled`	`bool`	Enables rhythmic multiplication/time-warp within bars (ekgun→dugun→tigun…); affects event timing density.
`layakari`	`LayakariState`	Carries the current/target laya and transition scheduling; interpreted per bar at render time.

Continued on next page

Field	Type	Meaning / Notes
`nadai_enabled`	`bool`	Enables subdivision-group accent warp (tisra/khanda/misra...); applied as a velocity emphasis pattern over 16 steps.
`nadai`	`NadaiState`	Holds the current nadai selection and related transition state (if any).

The remaining configuration fields (filter/DSP, sample placement, pattern presets, per-bar overrides) are shared across western and eastern modes; the eastern layer composes with these rather than replacing them.

8.9 Music Theory Primer for Parameterised Generation

The generator exposes musical concepts as user-facing parameters (key, scale, mood, energy curve) rather than as low-level note events. Consequently, a compact, implementation-oriented music theory vocabulary is useful to explain how scale choices, rhythmic emphasis, and harmonic density interact with the engine.

Pitch, Key, and Scales All pitched content is derived from a root pitch class (key) and a scale defined as semitone offsets within the octave. A scale thus becomes a constraint function: it filters candidate MIDI pitches (or oscillator frequencies) down to a musically coherent subset.

In Western terms, common scale families include major (Ionian), natural minor (Aeolian), and modal rotations (Dorian, Phrygian, Mixolydian). In eastern terms, the project also includes raga-coloured interval sets that emphasise characteristic seconds and sixths. Because the engine operates primarily in 12-TET, these are approximations; microtonal extensions (shruti offsets) are treated as future work (section 8).

Harmony and Implied Progression Psytrance often relies on implied harmony rather than functional chord progressions: bass + lead motion can suggest a tonal center even when no triads are voiced. Within the engine, harmony is therefore modelled as a lightweight layer:

- A scale constrains pitch selection.
- A degree-weighting profile biases landings toward stable tones (root, fifth) at low energy and toward colour tones (second, fourth, sixth) at higher energy.
- Energy can modulate harmonic density by enabling additional voices or widening pitch ranges.

This approach is compatible with both Western modes and raga-inspired sets, and avoids over-specifying chord symbols that are uncommon in the genre.

Rhythm, Meter, and Accent The system renders onto a fixed 4/4 transport grid with 16-step subdivision, but rhythm is perceived primarily through accent patterns. Classic EDM emphasis (beats 1 and 3) is augmented by syncopation styles and by tala-derived accent cycles (section 8). As a result, the engine can create the illusion of longer cycles (e.g., 7 or 10) while remaining compatible with DAW timelines and MIDI export.

Form and Energy Arrangement is expressed as an energy function over bars. Macro energy selects sections (intro, build, peak, breakdown), meso energy creates breathing, and micro energy controls per-hit density. Music theory concepts connect directly to this model:

- Lower energy favours fewer notes, narrower ranges, and stronger tonic anchoring.
- Higher energy supports faster note rates, wider ranges, and more dissonant colour tones.
- Cadences (e.g., tihai-like resolutions) provide audible section boundaries even without explicit chord changes.

9 Ableton Integration

The generator integrates with Ableton Live through three complementary mechanisms: MIDI file export, real-time OSC clip pushing, and Ableton Link transport synchronisation.

9.1 MIDI Export

The `midi` module generates Standard MIDI Files (SMF Type 1, 960 PPQ) using the `midly` crate. Each non-muted element is written as a separate track. Drum elements use General MIDI standard note numbers (kick = 36/C1, closed hi-hat = 42/F#1, clap = 39/D#1). Bass notes are converted from frequency to MIDI pitch via $p = 69 + 12 \log_2(f/440)$.

9.2 OSC Bridge

The `osc` module provides a bidirectional UDP bridge to Ableton Live via the AbletonOSC remote script [4], which exposes Live's Python API over OSC on ports 11000 (send) and 11001 (receive).

9.2.1 Architecture

The OSC subsystem consists of three components:

- `OscClient` — a `UdpSocket` wrapper that encodes and sends OSC messages using the `rosc` crate.
- `start_listener` — a background thread that binds to port 11001 and decodes incoming OSC responses, forwarding them through an `mpsc` channel.
- `AbletonBridge` — a high-level command API for clip management (`create_clip`, `add_notes`, `fire_clip`), scene control (`create_scene`, `fire_scene`), and transport (`start_playing`, `stop_playing`).

9.2.2 Clip Push Workflow

When the user triggers a clip push (or when a re-render occurs with an active OSC connection), the bridge executes the following sequence per element:

1. Delete existing clip in slot 0 of the target track.
2. Wait 20 ms (Ableton requires a brief pause after clip deletion).
3. Create a new clip of the appropriate length in beats.
4. Set the clip name to `Element – MoodName`.
5. Add all note events with pitch, start beat, duration, and velocity.

Track assignment is resolved by case-insensitive matching of Ableton track names against the element names ("kick", "bass", "hihat", "clap").

9.3 Ableton Link

The `link` module wraps the `rusty_link` crate [5]—a Rust FFI binding to Ableton's C++ Link library—to provide tempo and phase synchronisation with any Link peer (Ableton Live, other Link-enabled applications, or hardware).

9.3.1 Tempo Synchronisation

The `LinkBridge` registers callbacks via `set_tempo_callback` and `set_num_peers_callback` that forward events through an `mpsc` channel, drained by the TUI event loop each frame. Tempo changes

from any peer propagate within one beat. A feedback-loop guard ignores tempo deltas smaller than 0.001 BPM.

9.3.2 Phase-Locked Playback

When Link peers are present, the generator schedules playback start at the next bar boundary (quantum = 4 beats). The `cpal` audio callback maps Link beat positions to buffer frame indices using the `HostTimeFilter`:

$$\text{pos}_{\text{frames}} = \text{offset}_{\text{start}} + (b_{\text{link}} - b_{\text{origin}}) \cdot \frac{f_s \cdot 60}{\text{BPM}} \tag{16}$$

where b_{link} is the current Link beat position and b_{origin} is the beat at which playback was initiated.

9.4 Scene Control

The energy model's arrangement sections (Intro, Build, Peak, Breakdown) are mapped to Ableton scenes. When `follow_energy` mode is enabled, the generator fires the appropriate scene automatically as the bar cursor advances through section boundaries, driven by beat callbacks from the OSC listener.

10 Terminal User Interface

The TUI is implemented with `ratatui` [6] and `crossterm`, running at a 50 ms frame cadence. It provides a state machine with six screens: Launch, Playback, StepEdit, SessionList, PresetList, and EnergyEditor.

10.1 Screen State Machine

- **Launch** — parameter configuration: mood selection, BPM/bars adjustment, theme browsing, key/scale overrides, filter/LFO/arp settings, sample directory selection.
- **Playback** — live visualisation of pattern grids (4-bar view with per-step markers), energy curve sparkline, waveform display, element mute/solo/density controls, OSC/Link status indicators, and real-time parameter knobs.
- **StepEdit** — per-bar step editing with toggle-able 16th-note grid cells. Overridden patterns bypass the generative algorithm for the edited bar.
- **SessionList / PresetList** — browsable lists with timestamps and metadata, supporting load, save, and delete operations.
- **EnergyEditor** — direct manipulation of macro curve control points.

10.2 Re-Render Protocol

Parameter changes trigger a full re-render: the TUI stops playback (recording the current bar position), constructs a new `Generator`, calls `render()`, and resumes playback from the same bar offset. At 18 ms per render (section 11), this produces an imperceptible gap in audio output.

10.3 Theme System

Twelve theme presets are defined as a static array, each bundling a mood, BPM, key, scale, energy curve, and DSP parameters into a coherent sonic identity. Themes range from classic psytrance (Dark Phrygian, Acid Machine) through uplifting variants (Euphoric Rise, Solar Flare) to experimental settings (Alien Signal, Hitech Assault). Applying a theme overwrites all relevant fields in the `GeneratorConfig` via `apply_to()`.

11 Evaluation

We evaluate the generator on two axes: rendering throughput and output quality characteristics. All benchmarks were performed on a workstation with an Apple M2 processor and 16 GB of RAM, running Rust 1.82 on macOS 15.

11.1 Rendering Performance

Table 7 reports rendering times for tracks of increasing length. All renders use the *dark* mood preset at 142 BPM with default DSP parameters and no sample blending (pure synthesis).

The sub-linear scaling (real-time ratio *improves* with track length) reflects the fixed overhead of energy model initialisation and DSP state allocation, which is amortised over longer renders.

> **Sample Blending Overhead**
>
> When sample blending is active, render times increase by approximately 15–25% due to the additional PCM resampling and gain staging. A 64-bar track with full sample blending renders in approximately 22 ms—still well within the 50 ms TUI frame budget.

11.2 Signal Characteristics

The generator produces output with the following signal properties at default settings:

- **Peak level**: −0.5 dBFS (after normalisation and limiting).
- **RMS level**: approximately −12 to −8 dBFS during peak energy sections, consistent with mastered electronic music.
- **Frequency range**: 30 Hz to 20 kHz, bounded by the master HPF and the Nyquist frequency.
- **Stereo field**: kick and bass centre-panned; hi-hat slightly right (pan 0.55); clap slightly left (pan 0.45).
- **Internal precision**: 64-bit floating-point throughout the signal path; 16-bit integer at WAV export.

12 Conclusion

This report has presented Psytrance Generator, an energy-driven audio synthesis engine that generates full psytrance arrangements from declarative parameter configurations. The key contribution is the four-level energy model (section 3) that replaces manual arrangement with a hierarchical abstraction: macro curves define the track arc, meso patterns add phrase-level breathing, micro grooves shape per-step accents, and sub-channel gating controls per-element entry and response characteristics.

The synthesis pipeline (section 4) implements genre-appropriate timbres—pitch-swept kick drums, filtered detuned bass, metallic hi-hats, multi-tap claps, and FM leads—all operating at 64-bit precision with per-sample filter modulation for artifact-free sweeps. The sample intelligence subsystem (section 6) extends the sonic palette by analysing, classifying, and blending audio samples with synthesised output based on the energy state.

The Ableton integration (section 9) bridges the gap between generative composition and professional production, enabling MIDI export, real-time OSC clip pushing, Link transport synchronisation, and energy-driven scene control.

Rendering performance (section 11) exceeds 5,000× real time for typical track lengths, enabling the TUI's re-render-on-change workflow where any parameter adjustment produces an updated mix in under 20 ms.

Future development directions include PolyBLEP anti-aliased oscillators for lead synthesis, wavetable position morphing via envelope and LFO modulation, a granular pad/atmosphere engine, and MIDI input for overriding generative patterns with live performance data.

References

[1] R. Bristow-Johnson, *Audio EQ cookbook*, 2021. [Online]. Available: https://www.w3.org/2011/audio/audio-eq-cookbook.html

[2] M. R. Schroeder, "Natural sounding artificial reverberation," *Journal of the Audio Engineering Society*, vol. 10, no. 3, pp. 219–223, 1962.

[3] rustfft contributors, *RustFFT: A mixed-radix FFT library*, 2024. [Online]. Available: https://crates.io/crates/rustfft

[4] D. Jones, *AbletonOSC: OSC interface for Ableton Live*, 2024. [Online]. Available: https://github.com/ideoforms/AbletonOSC

[5] rusty_link contributors, *Rust bindings for Ableton Link*, 2024. [Online]. Available: https://crates.io/crates/rusty_link

[6] Ratatui contributors, *Ratatui: A Rust library for terminal user interfaces*, 2024. [Online]. Available: https://ratatui.rs/

Table 5. Implemented eastern layer features: config fields, implementation sites, and effects.

Feature	Config / Entry Point	Implementation + Effect
Raga scales + pakad	`Key.scale` (mood / override)	`src/theory/scales.rs`: interval set + optional pakad phrases constrain pitch selection and provide phrase seeds.
Pakad melody	implicit via active scale pakad	`src/engine/pakad.rs`: phrase-bank bar generation with energy-scaled density/stride/variation.
Alankar runs	`alankar_enabled`, `alankar`	`src/engine/alankar.rs`: degree permutations emit fast raga-constrained runs; used as a high-energy lead mode.
Bol text input	TUI override workflow	`src/engine/bol_parser.rs` + `src/tui/app.rs`: parse bols to (step, vel, articulation) and write tom overrides for the next bar.
Shruti tuning	`shruti_purity`	`src/engine/generator.rs` + tuning functions: blend 12-TET to shruti offsets when available; applied at frequency computation.
Raga modulation	`modulation`	`src/engine/raga_modulation.rs`: schedule source→target transition; render-time scale blending introduces target-unique tones over bars.
Samay mood	`mood_name="samay"`	`src/theory/mood.rs`: time-of-day raga + energy curve selection; presets filter ranges accordingly.
Yoga/ambient curves	`energy_curve_override` / mood preset	`src/engine/energy.rs`: named macro curves shape arrangement density and timbral evolution.
Breath/chakra/arc	`breath_lfo_enabled`, `chakra_mode`, `performance_arc_mode`	`src/dsp/lfo.rs` + `src/theory/chakra.rs` + arc logic: interpretable macro modulation and feature gating layered atop energy.

Table 7. Rendering performance by track length (pure synthesis, 44.1 kHz stereo). The real-time ratio is computed as track duration / render time.

Bars	Duration (s)	Render Time (ms)	Real-Time Ratio
8	13.5	3	4,500×
16	27.0	5	5,400×
32	54.1	10	5,410×
64	108.2	18	6,011×
128	216.3	35	6,180×
256	432.7	68	6,363×